AI ENGINEERING JOURNEY

WEEK 1 | DAY 3 | FOUNDATIONS | JUL 8 2026

Transformer Architecture

The Engine Behind Every LLM

The Big Question: Yesterday embeddings gave tokens meaning as vectors. Today: how does the model understand *relationships* between tokens? How does it know "bank" in "river bank" is different from "bank" in "savings bank"? The answer is the **Transformer architecture** — specifically **self-attention**, the invention that changed everything in 2017.

Week 1 — Day Plan

Day	Date	Topic	Status
Day 1	Mon Jul 6	Tokenization	DONE
Day 2	Tue Jul 7	Embeddings	DONE
Day 3	Wed Jul 8	**Transformer Architecture**	YOU ARE HERE
Day 4	Thu Jul 9	Training Lifecycle	Upcoming
Day 5	Fri Jul 10	Scaling Laws + Prompt Engineering	Upcoming
Day 6	Sat Jul 11	Structured Output + Architect's Lens	Upcoming
Day 7	Sun Jul 12	Week 1 Project — Model Selection Dashboard	Upcoming

Topics Covered Today

#	Topic	Coverage
1	**The 2017 Paper**	'Attention Is All You Need' — why it replaced RNNs and changed AI forever
2	**Self-Attention Mechanism**	Q, K, V vectors — how every token attends to every other token
3	**Causal vs Bidirectional Attention**	GPT (left-only) vs BERT (full sentence) — why it matters
4	**Multi-Head Attention**	12-96 parallel heads; each specialises in syntax, coreference, negation

5	**Positional Encoding**	Sinusoidal and RoPE — teaching word order without recurrence
6	**Feed-Forward Layers**	4x expansion; where factual knowledge is stored in LLMs
7	**Residual Connections + LayerNorm**	Why 96-layer models can be trained at all
8	**GPT vs BERT vs T5**	Decoder / Encoder / Encoder-Decoder — right architecture for each task
9	**FAANG Architecture Choices**	Google, Meta, OpenAI, Microsoft — what they chose and why
10	**Architect's Lens**	3 decisions: task matching, layer/latency tradeoff, context as architecture
11	**Hands-On Exercise**	BERT contextualised embeddings — same word, different vectors per context

Section 1 — The Paper That Changed Everything

> **2017. Google Brain. "Attention Is All You Need."** — This single paper is the reason GPT-4, LLaMA, Claude, Gemini, and every major LLM exist today.

Before 2017, all language models used **RNNs (Recurrent Neural Networks)** — they read text one word at a time, left to right, maintaining a hidden state. Problems:

- Slow to train — sequential processing, cannot parallelise across GPUs
- Forget long-range context — a word 200 tokens ago barely influences the present
- Vanishing gradients — training 20+ layers was extremely difficult

The Transformer replaced all of that with one idea: **self-attention** — let every token look at every other token *simultaneously*. No recurrence. No sequential processing. Fully parallelisable on GPUs. The result: models could train on thousands of GPUs at once, on internet-scale data, and learn long-range dependencies that RNNs simply could not capture.

> **GPT, BERT, T5, LLaMA, Claude, Gemini — every major LLM today is a Transformer.**

Section 2 — Self-Attention: The Core Idea

The problem self-attention solves:

The word "bank" means different things in: "I deposited money at the bank" (financial) vs "We sat by the river bank" (geography). A simple embedding (Day 2) gives "bank" the same vector in both sentences. **Self-attention fixes this** — it produces a contextualised embedding that changes based on surrounding words.

How self-attention works — step by step:

For each token, the model computes three vectors:

```
Q = Query 'What am I looking for?'
K = Key 'What do I contain?'
V = Value 'What information do I pass forward?'

Attention(Q, K, V) = softmax( Q x K_transpose / sqrt(d_k) ) x V
```

Plain English version — 5 steps:

Step 1: Every token asks a question (Query): 'What context do I need to understand myself?'

Step 2: Every token broadcasts what it contains (Key): 'Here is what I am and what I represent'

Step 3: The dot product Q·K gives an attention score — how relevant each token is to the question

Step 4: Scores are normalised with softmax → attention weights that sum to 1.0

Step 5: The Value vectors are weighted by those scores → new contextualised embedding for each token

How "bank" resolves ambiguity in "river bank":

```
"bank" asks Q: "What kind of bank am I?"
"river" answers with K: "I am a river — a body of water"
Attention score Q(bank) . K(river) is HIGH
"bank" pulls in information from "river" via its Value vector
Result: "bank" now has a river-flavoured contextual embedding
NOT a financial-bank embedding
```

Section 3 — Causal (Masked) vs Bidirectional Attention

This is the key architectural difference between GPT and BERT — two families of models you will use throughout this course:

GPT — Decoder-only — Causal (Masked) Attention:

```
Sentence: "The cat sat on the mat"

"cat" can only see: "The cat" (past tokens only)
"sat" can only see: "The cat sat" (past tokens only)
"mat" can only see: "The cat sat on the" (past tokens only)

Each token is MASKED from seeing future tokens.
This is how autoregressive generation works:
predict the next token → use it as input → repeat.
```

BERT — Encoder-only — Bidirectional Attention:

```
Sentence: "The cat sat on the mat"

"cat" can see: "The [cat] sat on the mat" (full sentence)
"sat" can see: "The cat [sat] on the mat" (full sentence)
"mat" can see: "The cat sat on the [mat]" (full sentence)

Every token sees everything. Much richer understanding,
but cannot generate text (seeing the future would be cheating).
```

Real-world consequence: Google uses BERT-style models for Search — understanding a query requires reading it fully. OpenAI uses GPT-style for ChatGPT — generating a response requires predicting left-to-right. Same data, different architecture, different job.

Section 4 — Multi-Head Attention: 12+ Perspectives at Once

Single-head attention captures one type of relationship per layer. Multi-head attention runs **multiple attention operations in parallel**, each focusing on different aspects of language:

```
Input → split into H heads → each head runs its own Q, K, V → concatenate → project back

GPT-3: 12 heads x 12 layers = 144 attention operations total
GPT-4: ~96 heads x 96 layers = ~9,216 attention operations total
LLaMA-3: 32 heads x 32 layers = 1,024 attention operations total
```

What different heads specialise in (discovered by research, not programmed):

Head Type	What It Learns
Syntactic heads	Subject-verb agreement, noun-adjective relationships
Coreference heads	"it" refers to "the cat" from 5 tokens ago
Positional heads	Attending to the token immediately before or after
Rare word heads	Attending to unusual tokens for disambiguation
Negation heads	Identifying "not", "never", "no" and their scope
Long-range heads	Connecting concepts separated by hundreds of tokens

> No engineer designed these specialisations — they **emerged** from training on text. The model discovered that language has structural dimensions and dedicated different attention heads to track each one. This is emergent behaviour at scale.

Section 5 — Positional Encoding: Teaching Order Without Recurrence

The embedding layer produces the same vector for "bank" regardless of position. But position matters enormously: "dog bites man" is very different from "man bites dog".

The solution: add a positional signal to every token embedding.

```
Final token representation = Token Embedding + Positional Encoding

Original (2017) — Sinusoidal encoding:
Position 0: sin(0 / 10000^(2i/d)), cos(0 / 10000^(2i/d)) → unique wave pattern
Position 1: sin(1 / 10000^(2i/d)), cos(1 / 10000^(2i/d)) → different pattern
Position 2: sin(2 / 10000^(2i/d)), cos(2 / 10000^(2i/d)) → different again

Modern (RoPE) — Rotary Positional Embedding:
Used by: LLaMA, Mistral, GPT-NeoX, Qwen
Encodes position as a rotation of the embedding vector
Generalises better to long sequences and can be extended at inference time
```

Why positional encoding matters for architecture: The PE scheme baked into a model at training time determines how far context can extend. This is why LLaMA-3.1 went from 8K to 128K context — RoPE supports extensions via YaRN and NTK-scaling tricks at inference. Original sinusoidal PE cannot do this.

Section 6 — Feed-Forward Layers, Residual Connections, Layer Norm

Feed-Forward Network (FFN) — factual memory of the model:

```
FFN(x) = max(0, xW1 + b1)W2 + b2

Two linear projections with a ReLU or GELU activation between them.
The expansion ratio is typically 4x:
If model dimension = 768, FFN expands to 3,072 then contracts back to 768.
GPT-4 model dimension ~12,288 → FFN width ~49,152 neurons per layer.

Research finding: FFN layers act as key-value memories.
When you ask 'What is the capital of France?'
→ attention routes to FFN neurons storing 'France → Paris' associations.
→ factual knowledge lives in the FFN, relational reasoning lives in attention.
```

Residual Connections (Skip Connections):

```
output = LayerNorm( x + Attention(x) )
output = LayerNorm( x + FFN(x) )

The 'x +' part is the residual. It allows gradients to flow directly back
through the network during training without vanishing.
Without residuals, a 96-layer model would be impossible to train.
```

Layer Normalisation:

Normalises activations across the embedding dimension after each sub-layer. Keeps values in a stable range, prevents exploding/vanishing gradients, and makes training of deep networks dramatically more stable.

Section 7 — Full Transformer Stack: One Layer Then 96 of Them

One Transformer layer does this in sequence:

```
Input → Layer Norm → Multi-Head Attention → Residual Add
→ Layer Norm → Feed-Forward Network → Residual Add
→ Output (exactly same shape as input)
```

That output feeds into the next identical layer. A modern LLM stacks 32–96 of these.

Why depth matters — what each layer range learns:

Layer Range	What It Learns
Early layers (1-10)	Syntax, local patterns, part-of-speech, word boundaries
Middle layers (11-50)	Semantics, coreference resolution, named entities, word sense
Late layers (51-96)	Task-specific reasoning, world knowledge retrieval, long-range logic

Production insight: FAANG research shows you can prune 20-30% of layers from large models with minimal quality loss — layers are not all equally important for every task. This is the basis of model distillation and layer pruning techniques you will see in Week 2.

Section 8 — GPT vs BERT vs T5: Right Architecture for Each Task

Architecture	Type	Best For	Cannot Do	Examples
GPT / LLaMA	Decoder-only	Text generation, chat, code, reasoning, instruction following	Not ideal for classification without fine-tuning	GPT-4, LLaMA-3, Mistral, Claude
BERT / RoBERTa	Encoder-only	Classification, NER, search, sentence embeddings, QA (extractive)	Cannot generate new text	BERT, RoBERTa, DistilBERT, DeBERTa
T5 / BART	Enc-Decoder	Translation, summarisation, question answering, seq-to-seq tasks	Slower and heavier per token than decoder-only	T5, BART, mT5, FLAN-T5

Quick decision guide:

```
Task → Right Architecture
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Text generation / chat → Decoder-only (GPT, LLaMA)
Classification / NER → Encoder-only (BERT, RoBERTa)
Translation / summarisation → Encoder-Decoder (T5, BART)
Embeddings for RAG → Encoder-only or Bi-encoder
Code generation → Decoder-only (CodeLlama, GPT-4)
Semantic search → Encoder-only (bi-encoder architecture)
```

Section 9 — FAANG Architecture Choices in Production

Google — BERT for Search, Gemini for Generation

Google runs BERT-family models for understanding search queries — bidirectional attention reads the full query at once. They run Gemini (decoder-only) for AI Overviews and generation tasks. Two different architectures running in production for two different jobs.

Meta — LLaMA (Decoder-only) with Key Architectural Innovations

Meta standardised on the decoder-only GPT architecture for LLaMA 1/2/3. Their key architectural innovations: (1) RoPE positional encoding for better long context, (2) Grouped Query Attention (GQA) — reduces memory during inference by sharing key-value heads, (3) SwiGLU activation in FFN layers — replaces ReLU, better performance per parameter.

OpenAI — GPT-4 (Decoder-only, likely Mixture of Experts)

GPT-4 is believed to use a Mixture of Experts (MoE) variant: instead of one giant FFN, it has multiple 'expert' FFNs and a router that selects 2 of 8 (or similar) for each token. This gives the parameter count of a 1.8T model but the compute cost of a ~220B model per token.

Microsoft — Phi Series (Small Decoder-only, High-Quality Data)

Microsoft's Phi-3 (3.8B parameters) outperforms models 10x its size on benchmarks. Architecture insight: quality of training data can substitute for scale. Same decoder-only architecture, but trained exclusively on textbook-quality, curated synthetic data.

Section 10 — Architect's Lens: 3 Architecture Decisions in Production

Decision 1 — Match Architecture to Task (Not 'Biggest Model Wins')

Using GPT-4 for email classification wastes 10-100x more compute than BERT-base. Using BERT for a chatbot is impossible — it cannot generate text. Right architecture for the right task is the most impactful cost-quality decision you make in system design.

```
Task → Right Architecture
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Text generation / chat → Decoder-only (GPT, LLaMA)
Classification / NER → Encoder-only (BERT, RoBERTa)
Translation / summary → Encoder-Decoder (T5, BART)
Embeddings for RAG → Encoder-only or Bi-encoder
```

Decision 2 — Layer Count vs Inference Latency

More layers = more quality, but linearly more inference time and memory. A 96-layer model is roughly 3x slower than a 32-layer model at the same hidden dimension. For latency-sensitive applications (chatbots, real-time suggestions), you trade layers for speed.

```
LLaMA-3 8B (32 layers) → Real-time chat, mobile, edge, <100ms latency
LLaMA-3 70B (80 layers) → Batch analysis, high-quality output, 500ms+ latency OK
GPT-4 (~96 layers) → Complex reasoning, highest quality, 1-3s latency
```

Decision 3 — Context Window is an Architecture Property, Not a Config

The positional encoding scheme baked into a model at training time determines how far context can extend. You cannot take a model trained with 4K context and simply run it at 128K — the positional signals become meaningless beyond training length. If you need long context, choose a model whose architecture AND training support it.

```
Model → Context → PE Type
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
LLaMA-3.1 → 128K → RoPE (extended)
Claude Sonnet → 200K → Custom PE
Gemini 1.5 Pro → 1M → Custom PE
LLaMA-3 base → 8K → RoPE (base range)
```

Section 11 — Hands-On Exercise: Contextualised Embeddings with BERT

Prove to yourself that self-attention produces different embeddings for the same word in different contexts. This directly demonstrates what self-attention achieves.

Setup:

```
cd /Users/thaya/AI-Engineering/week-01/notebooks
source /Users/thaya/AI-Engineering/.venv/bin/activate
pip install transformers torch
```

Create file: week-01/notebooks/day3_transformer.py

```
from transformers import AutoTokenizer, AutoModel
import torch

# Load BERT — encoder-only, bidirectional
model_name = 'bert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Two sentences with the same word 'bank' in different contexts
sentences = [
'I deposited money at the bank yesterday.',
'We sat by the river bank and watched the water.',
]

for sentence in sentences:
inputs = tokenizer(sentence, return_tensors='pt')
with torch.no_grad():
outputs = model(**inputs)

tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])
bank_idx = tokens.index('bank')
bank_embed = outputs.last_hidden_state[0][bank_idx]

print(f'Sentence: {sentence}')
print(f' token index for bank: {bank_idx}')
print(f' embedding shape: {bank_embed.shape}')
print(f' first 5 values: {bank_embed[:5].tolist()}')
print()

# Expected: the two bank vectors are numerically DIFFERENT
# Self-attention changed the same word's representation based on context
```

What You Will Observe:

- The embedding for 'bank' in the financial sentence has different values than in the river sentence
- This is self-attention at work — surrounding tokens pulled the representation in different directions
- Compare the cosine similarity between the two 'bank' vectors — it will be ~0.7, not 1.0
- Try 'He gave her a present' vs 'She is present in the room' — the word 'present' will differ

Day 3 — Complete Summary

#	Topic	Key Takeaway	Status
1	**The 2017 Paper**	Attention Is All You Need replaced RNNs; enabled parallel GPU training at scale	**DONE**
2	**Self-Attention (Q,K,V)**	Every token attends to every other; resolves word sense ambiguity via context	**DONE**
3	**Causal vs Bidirectional**	GPT = left-only (generation); BERT = full sentence (understanding/classification)	**DONE**
4	**Multi-Head Attention**	12-96 parallel heads; each specialises: syntax, coreference, negation, long-range	**DONE**
5	**Positional Encoding**	Sinusoidal (2017) or RoPE (modern); RoPE enables long-context extensions	**DONE**
6	**FFN Layers**	4x expansion; factual knowledge storage; SwiGLU outperforms ReLU	**DONE**
7	**Residual + LayerNorm**	Skip connections enable 96-layer training; LayerNorm stabilises activations	**DONE**
8	**GPT vs BERT vs T5**	Decoder=generation; Encoder=understanding; Enc-Dec=seq-to-seq; match to task	**DONE**
9	**FAANG Choices**	Google (BERT+Gemini), Meta (RoPE+GQA), OpenAI (MoE), Microsoft (Phi/data quality)	**DONE**
10	**Architect's Lens**	Match arch to task; layer count = latency tradeoff; context = architecture property	**DONE**
11	**Hands-On Exercise**	BERT contextualised 'bank' embeddings — same word, different vectors per context	**ACTION**

Preview — Day 4: Training Lifecycle (Thursday, Jul 9)

Now that you understand the Transformer architecture, the next question is: **how is a model actually trained?** What happens between raw internet text and a model that can write code, answer questions, and reason?

Day 4 will cover:

- Pre-training: next-token prediction on trillions of tokens of internet data
- Supervised Fine-Tuning (SFT): teaching the model to follow instructions
- RLHF: Reinforcement Learning from Human Feedback — aligning to human preferences
- DPO: Direct Preference Optimisation — the simpler modern alternative to RLHF
- The training compute equation: tokens × parameters × 6 FLOPs per token
- Architect's Lens: training cost estimation, choosing pre-trained base models

AI Engineering Journey — Week 1 Day 3 | July 8 2026 | Instructor: Pro AI Principal Architect | github.com/irudi2021/ai-engineering-journey